

**SMART ENVIRONMENT MONITORING AND ALERT
SYSTEM**

Internship Assignment Submission

Course

Internet of Things (IoT)

Submitted By

THILOOKSHAN.S

TASK 1: INTRODUCTION TO IOT & DEVICE COMMUNICATION

Goal

The goal of this task is to understand the basic concept of the Internet of Things, also called IoT, and how devices communicate with each other.

IoT is a technology concept where physical devices are connected to the internet or a network so they can collect data, send data, receive commands, and perform automatic actions. These devices can include sensors, cameras, smart lights, smart meters, wearable devices, machines, and home appliances.

In an IoT system, devices do not work alone. They communicate with other devices, cloud platforms, dashboards, and users. This communication allows real-world information to be monitored and controlled using software.

What Is IoT?

The Internet of Things is a network of physical objects that are connected through the internet. These objects are called "things" because they can be everyday devices such as a fan, light bulb, car, watch, water pump, or temperature sensor.

IoT devices usually have sensors to collect information from the environment. After collecting data, the device sends it to another system for processing. Based on the result, the system may display information to a user or trigger an action automatically.

For example, a smart home temperature sensor can measure room temperature. If the temperature becomes too high, the system can automatically turn on a fan or air conditioner.

IoT Architecture

An IoT system usually has four main layers:

1. Device Layer

The device layer contains sensors and actuators.

Sensors collect data from the environment. Examples include:

- Temperature sensors
- Humidity sensors
- Motion sensors
- Light sensors
- Gas sensors
- Soil moisture sensors

Actuators perform actions based on commands. Examples include:

- Motors
- Fans
- Water pumps
- Smart lights
- Buzzers
- Door locks

2. Network Layer

The network layer is responsible for sending data from devices to other systems. IoT devices can communicate using different network technologies.

Common communication methods include:

- Wi-Fi
- Bluetooth
- Zigbee
- LoRaWAN
- Ethernet
- Cellular networks

This layer helps devices send sensor readings to cloud platforms, mobile apps, or local servers.

3. Cloud or Processing Layer

The cloud layer stores, processes, and analyzes IoT data. It can check sensor readings, apply rules, and make decisions.

For example, if a temperature reading is above 30 degrees Celsius, the cloud system can identify it as a high-temperature condition and send a command to turn on a cooling fan.

The cloud layer may also store historical data so users can view trends and reports later.

4. Application Layer

The application layer is where users interact with the IoT system. This can be a mobile app, web dashboard, or desktop application.

The application displays sensor readings, alerts, graphs, and device status. Users can also control devices from the application.

For example, a smart home app can show whether lights are on or off and allow the user to switch them manually.

How Devices Communicate in IoT

IoT devices communicate by sending data messages through a network. A sensor first collects data from the environment. Then a microcontroller or processor prepares the data and sends it to another system.

A common data flow is:

1. Sensor collects data.
2. Microcontroller reads the sensor value.
3. Device sends data through a network.
4. Cloud platform receives and stores the data.
5. Software analyzes the data.
6. Dashboard displays the result.
7. If needed, a command is sent to an actuator.
8. The actuator performs an action.

Example:

A humidity sensor in a greenhouse measures humidity. The sensor sends the value to a controller such as an ESP32. The ESP32 sends the data to the cloud using Wi-Fi. The cloud checks the humidity level. If the humidity is too low, the system sends a command to start a water pump.

Real-World IoT Use Cases

1. Smart Home

Smart home systems use IoT devices such as motion sensors, smart lights, thermostats, and door locks. These devices help automate home activities and improve security.

Example:

When a motion sensor detects movement at night, the system can automatically turn on a light.

2. Smart Agriculture

IoT is used in agriculture to monitor soil moisture, temperature, humidity, and weather conditions. Farmers can use this data to improve crop growth and reduce water waste.

Example:

A soil moisture sensor can detect dry soil and automatically turn on irrigation.

3. Healthcare Monitoring

IoT devices are used in healthcare to monitor patient health. Wearable devices can track heart rate, oxygen level, body temperature, and movement.

Example:

If a patient's heart rate becomes abnormal, the system can send an alert to a doctor or caregiver.

4. Industrial Monitoring

Factories use IoT sensors to monitor machines, temperature, pressure, vibration, and energy usage. This helps detect problems early and avoid machine failures.

Example:

A vibration sensor can detect unusual machine movement and alert technicians before the machine breaks down.

5. Smart City

Smart cities use IoT for traffic management, air-quality monitoring, smart parking, street lighting, and waste management.

Example:

Smart street lights can turn on only when people or vehicles are nearby, saving electricity.

Data Flow in an IoT System

Example system: Smart Greenhouse Monitoring System

1. Temperature and humidity sensors collect environmental data.
2. A microcontroller such as an ESP32 reads the sensor values.
3. The microcontroller sends data to the cloud through Wi-Fi.
4. The cloud stores the readings in a database.
5. The system checks whether the readings are within safe limits.
6. A dashboard displays temperature, humidity, and alert status.
7. If the temperature is too high, the system sends a command to turn on a fan.
8. If the soil is dry, the system sends a command to turn on a water pump.

This shows how data moves from the physical environment to software and then back to physical devices through automation.

TASK 2: SENSOR DATA SIMULATION

Goal

The goal of this task is to simulate sensor data collection using software or basic hardware. In this task, sensor values such as temperature, humidity, and motion are generated, displayed, and logged.

This helps us understand how IoT devices collect data from the environment and prepare it for monitoring or automation.

What Is Sensor Data Simulation?

Sensor data simulation means creating artificial sensor readings using software instead of using real physical sensors. This is useful when hardware is not available or when we want to test an IoT system before connecting real devices.

For example, instead of using a real temperature sensor, a program can generate random temperature values such as 26 °C, 28 °C, or 31 °C. These values can be displayed on a screen and stored in a log just like real sensor data.

Sensors Used in This Simulation

This task simulates three types of sensor data:

1. Temperature Sensor

A temperature sensor measures heat in the environment.

Example values:

- 25 °C
- 28 °C
- 31 °C
- 35 °C

In a real IoT system, this sensor can be used in smart homes, greenhouses, cold storage rooms, and factories.

2. Humidity Sensor

A humidity sensor measures the amount of moisture in the air.

Example values:

- 50%
- 62%
- 70%
- 82%

Humidity data is useful in agriculture, weather monitoring, storage rooms, and indoor air-quality systems.

3. Motion Sensor

A motion sensor detects movement.

Example values:

- Motion detected
- No motion detected

Motion sensors are commonly used in security systems, smart lighting, and automatic doors.

Simulation Method

The simulation can be created using a simple software program. The program generates random values for each sensor.

The program should:

1. Generate a random temperature value.
2. Generate a random humidity value.
3. Randomly decide whether motion is detected or not.
4. Display the generated sensor values.
5. Store or log the generated values with time.

Example Sensor Data

Time	Temperature	Humidity
10:00:01 AM	27 C	60%
10:00:03 AM	29 C	64%
10:00:05 AM	31 C	68%
10:00:07 AM	33 C	72%
10:00:09 AM	28 C	59%

This table shows how sensor values can change over time. Each row represents one sensor reading collected by the system.

```
import random
import time
from datetime import datetime

print("IoT Sensor Simulation Started")
print("-----")

for i in range(10):
    temperature = random.randint(25, 40)
    humidity = random.randint(50, 90)
    current_time = datetime.now().strftime("%I:%M:%S %p")

    print(f"Time: {current_time}")
    print(f"Temperature: {temperature} °C")
    print(f"Humidity: {humidity} %")
    print("-----")

    time.sleep(1)

print("IoT Sensor Simulation Finished")
```

Sample python Code

```
IoT Sensor Simulation Started
-----
Time: 08:55:06 PM
Temperature: 32 °C
Humidity: 61 %
-----
Time: 08:55:07 PM
Temperature: 27 °C
Humidity: 57 %
-----
Time: 08:55:08 PM
Temperature: 32 °C
Humidity: 57 %
-----
Time: 08:55:09 PM
Temperature: 38 °C
Humidity: 63 %
-----
Time: 08:55:10 PM
Temperature: 31 °C
Humidity: 66 %
-----
Time: 08:55:11 PM
Temperature: 26 °C
Humidity: 84 %
...
Temperature: 32 °C
Humidity: 78 %
-----
IoT Sensor Simulation Finished
```

Displaying Sensor Values

TASK 3: IOT DATA MONITORING DASHBOARD

Goal

The goal of this task is to create a simple web dashboard to display IoT sensor data. A dashboard helps users monitor sensor readings clearly and understand the current condition of an IoT system.

In this task, the dashboard displays temperature and humidity values. The values update automatically, and the readings are shown using cards and a table.

What Is an IoT Data Monitoring Dashboard?

An IoT data monitoring dashboard is a web page or application that displays live sensor readings from IoT devices.

In real IoT systems, sensors collect data from the environment and send it to a server or cloud platform. The dashboard receives that data and displays it to the user.

For this task, real sensors are not required. The dashboard can use simulated sensor data.

Dashboard Features

The web dashboard includes:

- Live temperature reading
- Live humidity reading
- Current time
- Status message
- Sensor data table
- Automatic update every second

Sensor Readings Used

1. Temperature

Temperature shows how hot or cold the environment is.

Example:

Temperature: 30 °C

2. Humidity

Humidity shows the amount of moisture in the air.

Example:

Humidity: 65%

Status Logic

The dashboard checks the sensor values and shows a status.

Rules:

- If temperature is greater than 35 C, status is Alert.
- If humidity is greater than 80%, status is Alert.
- Otherwise, status is Normal.

Sample Dashboard Code and result

```
import random
import time
from datetime import datetime

# Store sensor logs
sensor_logs = []

print("=====")
print("  IoT Data Monitoring Dashboard")
print("=====")

def generate_sensor_data():
    # Generate random sensor values
    temperature = random.randint(25, 40)
    humidity = random.randint(50, 90)

    # Get current time
    current_time = datetime.now().strftime("%H:%M:%S")

    # Check system status
    status = "Normal"

    if temperature > 35 or humidity > 80:
        status = "Alert"

    # Store data
    sensor_logs.append({
        "time": current_time,
        "temperature": temperature,
        "humidity": humidity,
        "status": status
    })
```

```
# Display dashboard
print("\n-----")
print(f"Time       : {current_time}")
print(f"Temperature : {temperature} °C")
print(f"Humidity    : {humidity} %")
print(f>Status     : {status}")
print("-----")

# Run dashboard continuously
while True:
    generate_sensor_data()
    time.sleep(1)
```

```
...
=====
IoT Data Monitoring Dashboard
=====

-----
Time       : 21:56:31
Temperature : 27 °C
Humidity    : 67 %
Status     : Normal
-----

-----
Time       : 21:56:32
Temperature : 26 °C
Humidity    : 61 %
Status     : Normal
-----

-----
Time       : 21:56:33
Temperature : 32 °C
Humidity    : 81 %
Status     : Alert
-----

...
Temperature : 25 °C
Humidity     : 73 %
Status       : Normal
-----
```

Web Dashboard Code and result

```
1 import streamlit as st
2 import pandas as pd
3 import random
4 from datetime import datetime
5 import time
6
7 # Page configuration
8 st.set_page_config(
9     page_title="IoT Monitoring Dashboard",
10     layout="wide"
11 )
12
13 # Dashboard title
14 st.title("🌐 IoT Data Monitoring Dashboard")
15
16 # Initialize session state
17 if "data" not in st.session_state:
18     st.session_state.data = pd.DataFrame(
19         columns=["Time", "Temperature", "Humidity", "Status"]
20     )
21
22 # Generate sensor values
23 temperature = random.randint(25, 40)
24 humidity = random.randint(50, 90)
25
26 # Current time
27 current_time = datetime.now().strftime("%H:%M:%S")
28
29 # Automation logic
30 status = "Normal"
31
32 if temperature > 35 or humidity > 80:
33     status = "Alert"
34
35 # Create dashboard cards
36 col1, col2, col3 = st.columns(3)
37
38 col1.metric(
39     label="🌡 Temperature",
40     value=f"{temperature} °C"
41 )
42
43 col2.metric(
44     label="💧 Humidity",
45     value=f"{humidity} %"
46 )
47
48 col3.metric(
49     label="🟡 Status",
50     value=status
51 )
52
53 # Add new sensor data
54 new_data = pd.DataFrame({
55     "Time": [current_time],
56     "Temperature": [temperature],
57     "Humidity": [humidity],
58     "Status": [status]
59 })
60
61 # Update dataframe
62 st.session_state.data = pd.concat(
63     [new_data, st.session_state.data],
64     ignore_index=True
65 )
66
67 # Keep latest 20 rows
68 st.session_state.data = st.session_state.data.head(20)
69
70 # Chart section
71 st.subheader("📊 Live Sensor Graph")
```

```
72
73 chart_data = st.session_state.data.iloc[::-1]
74
75 chart_data = chart_data[
76     ["Temperature", "Humidity"]
77 ]
78
79 st.line_chart(chart_data)
80
81 # Table section
82 st.subheader("📄 Sensor Data Table")
83
84 st.dataframe(
85     st.session_state.data,
86     use_container_width=True
87 )
88
89 # Auto refresh
90 time.sleep(1)
91 st.rerun()
```



IoT Data Monitoring Dashboard

Temperature

29 °C

Humidity

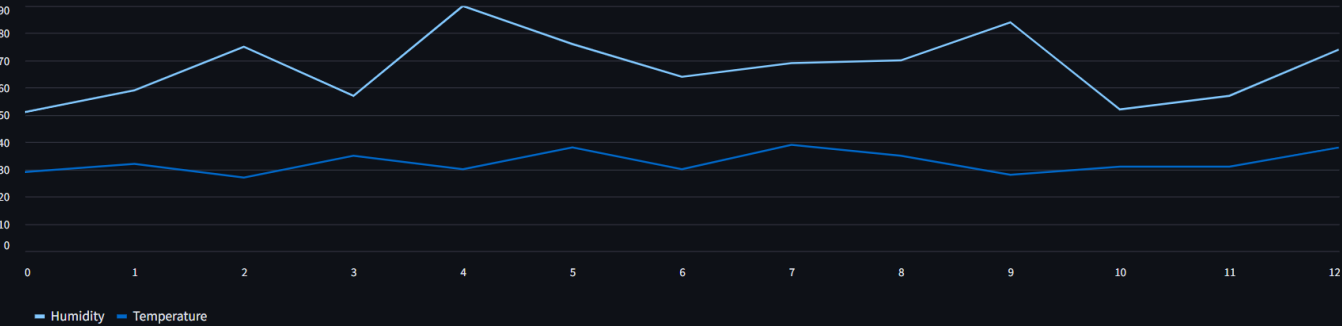
51 %

Status

Normal



Live Sensor Graph



Sensor Data Table

	Time	Temperature	Humidity	Status
3	22:06:28	35	57	Normal
4	22:06:26	30	90	Alert
5	22:06:25	38	76	Alert
6	22:06:24	30	64	Normal
7	22:06:22	39	69	Alert
8	22:06:21	35	70	Normal
9	22:06:20	28	84	Alert
10	22:06:18	31	52	Normal
11	22:06:17	31	57	Normal
12	22:06:15	38	74	Alert

TASK 5: IOT MINI PROJECT

Project Title

Smart Environment Monitoring and Alert System

Project Goal

The goal of this mini project is to build a small IoT-based system that combines the concepts learned in the previous tasks.

This project simulates environmental sensor data, displays the data on a dashboard, applies automation rules, and shows alerts when readings become unsafe.

Project Description

The Smart Environment Monitoring and Alert System is a simple IoT project that monitors:

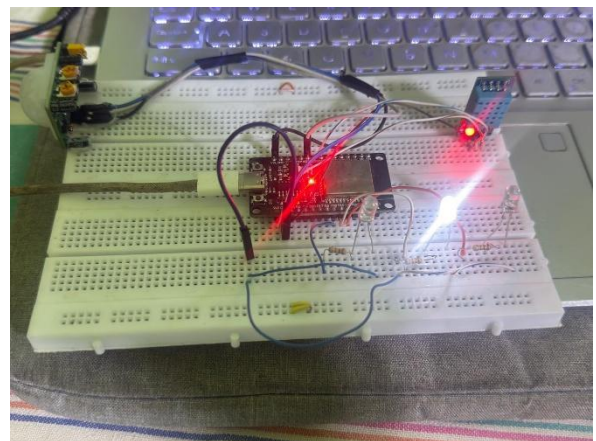
- Temperature
- Humidity
- Motion

The system generates sensor readings using software simulation. These readings are displayed in a dashboard. The system also checks the readings using automation rules.

1. If the temperature is high, the cooling fan is turned on.
2. If the humidity is high, ventilation is turned on.
3. If motion is detected, the security light is turned on.

Hardware Connections and setup

Component	ESP32 Pin
DHT11 DATA	GPIO 26
PIR Sensor OUT	GPIO 27
Fan LED	GPIO 18
Ventilation LED	GPIO 19
Security LED	GPIO 21
DHT11 VCC	3.3V
DHT11 GND	GND
PIR VCC	5V
PIR GND	GND



System Working Principle

Temperature Monitoring

The DHT11 sensor continuously measures temperature.

If temperature exceeds 35 C:

- Cooling Fan LED turns ON.

Humidity Monitoring

The DHT11 sensor also measures humidity.

If humidity exceeds 80%:

- Ventilation LED turns ON.

Motion Detection

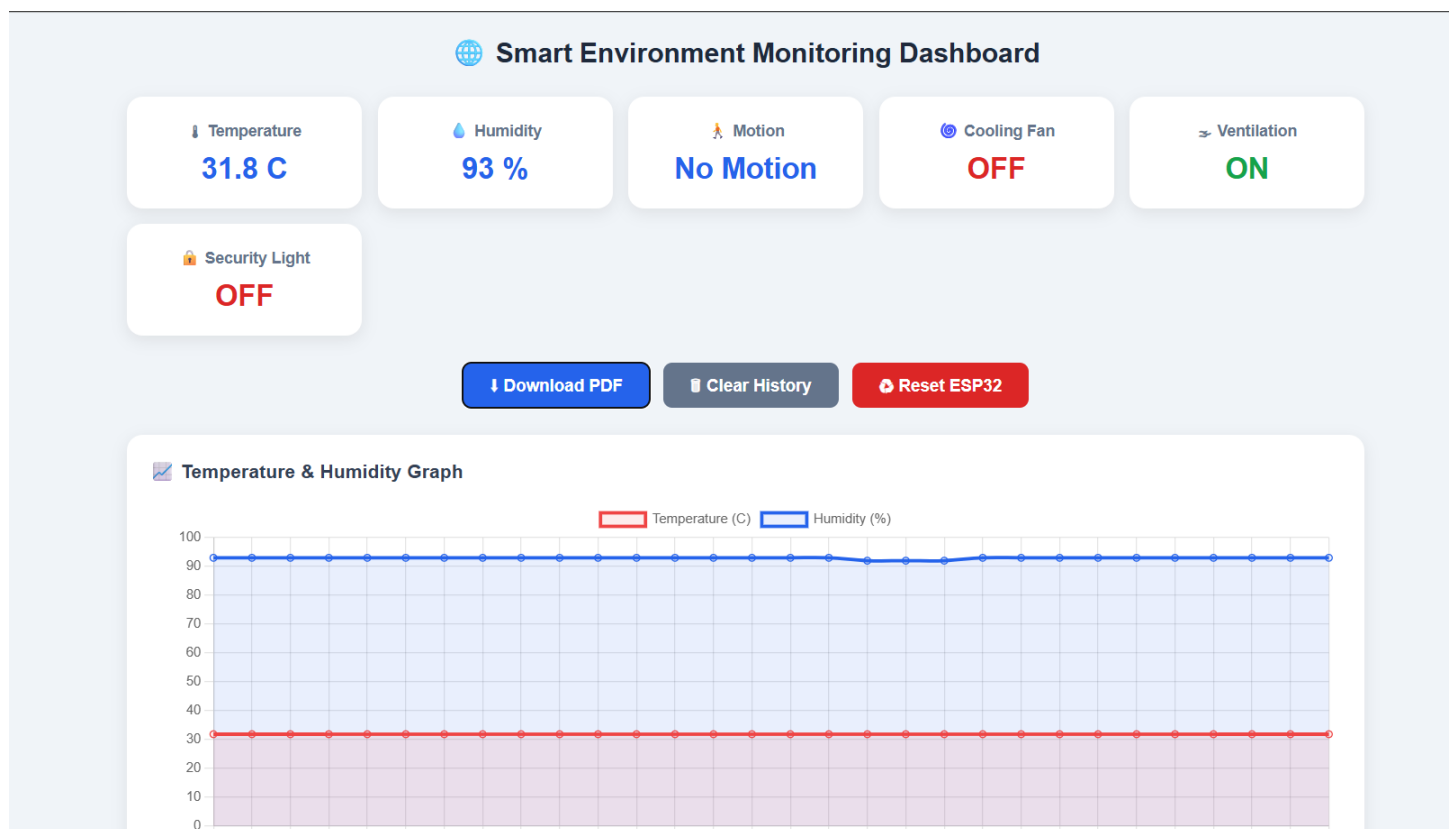
The PIR sensor detects human motion.

If motion is detected:

- Security LED turns ON.

The ESP32 processes all sensor data and sends the information to a web dashboard through Wi-Fi.

Web Dashboard



 Data History Table

Time	Temp (C)	Humidity (%)	Motion	Fan	Ventilation	Security
11:47:28 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:26 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:25 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:24 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:23 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:22 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:21 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:20 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:19 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:18 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:17 AM	31.8	92	No Motion	OFF	ON	OFF
11:47:16 AM	31.8	92	No Motion	OFF	ON	OFF
11:47:16 AM	31.8	92	No Motion	OFF	ON	OFF
11:47:14 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:13 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:12 AM	31.8	93	No Motion	OFF	ON	OFF
11:47:12 AM	31.8	93	No Motion	OFF	ON	OFF

Procedure

1. Connect all sensors and LEDs to ESP32.
2. Install ESP32 board package in Arduino IDE.
3. Install the DHT sensor library.
4. Replace the Wi-Fi name and password in the program.
5. Upload the Arduino program.
6. Open Serial Monitor.
7. Copy the IP address shown in Serial Monitor.
8. Open the IP address in a web browser.
9. Observe live sensor data and automation status.

Results

The system successfully:

- Monitored environmental conditions.
- Displayed real-time data on a web dashboard.
- Performed automatic control actions.
- Detected motion using PIR sensor.
- Demonstrated IoT communication through Wi-Fi.

Conclusion

The Smart Environment Monitoring and Alert System was successfully developed using ESP32. The project demonstrated real-time IoT monitoring, web-based visualization, sensor integration, and basic automation.

This project improved understanding of:

- IoT systems
- Embedded programming
- ESP32 microcontroller
- Sensor interfacing
- Web dashboard development
- Automation logic

The system can be further expanded with cloud storage, mobile applications, email or SMS alerts, relay-controlled appliances, and MQTT communication.